# NETMORPH

## Dossier Technique de Production

Version 3.0 — Architecture SaaS (Flask)

Plateforme de migration cross-vendor de configurations firewall

Projet Tutoré — IUT de Valence
BUT Réseaux & Télécommunications
Mars 2026

*Document CONFIDENTIEL — Usage interne*

# Table des matières


**1. Présentation générale** ..... 3

1.1 Objet du document ..... 3

1.2 Qu'est-ce que NETMORPH ? ..... 3

1.3 Évolution V2 → V3 ..... 4

**2. Architecture technique** ..... 5

2.1 Vue applicative ..... 5

2.2 Vue infrastructure ..... 6

2.3 Vue réseau et sécurité du déploiement ..... 6

2.4 API REST ..... 7

2.5 Gestion des sessions ..... 7

**3. Guide de déploiement** ..... 8

3.1 Prérequis ..... 8

3.2 Installation pas à pas ..... 8

3.3 Configuration Nginx + HTTPS ..... 9

3.4 Lancement de l'application ..... 9

3.5 Configuration de l'IA (optionnel) ..... 10

**4. Guide de maintenance** ..... 11

4.1 Logs et monitoring ..... 11

4.2 Mise à jour du code ..... 11

4.3 Sauvegarde ..... 12

4.4 Troubleshooting courant ..... 12

**5. Analyse de risques — EBIOS RM** ..... 13

5.1 Atelier 1 — Cadrage et socle de sécurité ..... 13

5.2 Atelier 2 — Sources de risque ..... 15

5.3 Atelier 3 — Scénarios stratégiques ..... 16

5.4 Atelier 4 — Scénarios opérationnels ..... 17

5.5 Atelier 5 — Traitement du risque ..... 18

**6. Annexes** ..... 20

# 1. Présentation générale

## 1.1 Objet du document

Ce document est destiné à l'équipe de production qui reprendra le développement et la maintenance de NETMORPH. Il décrit l'architecture logicielle et infrastructure, les procédures de déploiement et de maintenance, ainsi que l'analyse de risques de sécurité selon la méthodologie EBIOS Risk Manager de l'ANSSI.

L'objectif est de permettre à une équipe tierce de comprendre le produit, de le déployer sur une nouvelle infrastructure et de le maintenir en conditions opérationnelles sans intervention de l'équipe de développement initiale.

## 1.2 Qu'est-ce que NETMORPH ?

NETMORPH est une plateforme SaaS de migration et de gestion de configurations firewall. Elle permet aux administrateurs réseau de convertir automatiquement des configurations entre équipements de marques différentes, via une technique de traduction sémantique des politiques de sécurité (cross-vendor policy translation).

Fonctionnalités principales :

- Parsing et visualisation de configurations XML (dashboard interactif)
- Éditeur rapide : création de règles de filtrage, interfaces réseau, VLANs
- Migration cross-vendor bidirectionnelle WatchGuard Fireware ↔ pfSense
- Audit de sécurité automatique (détection de règles permissives, ports dangereux, services non chiffrés)
- Auto-réparation XML via IA générative (Google Gemini) en cas d'erreur de structure post-migration
- Authentification et sessions isolées par utilisateur

Vendors supportés :

| Vendor | Format | Direction |
|---|---|---|
| WatchGuard Fireware | XML (balise racine <profile>) | Source et destination |
| pfSense | XML (balise racine <pfsense>) | Source et destination |

## 1.3 Évolution V2 → V3

| Critère | V2 (Desktop) | V3 (SaaS) |
|---|---|---|
| Framework | PyQt6 + QWebEngineView | Flask + Gunicorn + Nginx |
| Protocole IPC | netmorph:// (URL propriétaire) | API REST JSON (fetch → Flask) |

| | | |
|---|---|---|
| Déploiement | Exécutable Windows (.exe) | VM OpenStack (Ubuntu) |
| Multi-utilisateurs | Non (mono-poste) | Oui (sessions isolées UUID) |
| Accès | Installation locale requise | Navigateur web uniquement |
| Sécurité | Isolation locale | HTTPS + authentification + sessions |
| Stockage | Fichiers locaux | Uploads temporaires (purge 24h) |

# 2. Architecture technique

## 2.1 Vue applicative

L'architecture suit le pattern MVC (Modèle-Vue-Contrôleur) adapté à Flask. Le projet est composé de 6 modules Python interdépendants :

| Module | Rôle | Lignes |
|---|---|---|
| app.py | Point d'entrée Flask, routes HTTP, authentification, sessions | ~340 |
| engine.py | Moteur de migration WG↔pfSense, injection règles/interfaces, IA | ~1850 |
| parser.py | Parsing XML des configurations, extraction données dashboard | ~150 |
| ui_template.py | Génération HTML/CSS/JS de l'interface complète | ~2275 |
| audit.py | Audit de sécurité automatique, scoring des règles | ~380 |
| utils.py | Helpers : chemins, sessions thread-local, fichiers | ~50 |

Chaîne de communication :

```
Client (navigateur)
    ↓ HTTPS (port 443)
Nginx (reverse proxy + SSL termination)
    ↓ HTTP (127.0.0.1:5000)
Gunicorn (2 workers WSGI)
    ↓
Flask app (app.py → engine/parser/audit/ui_template)
```

## 2.2 Vue infrastructure

| Composant | Spécification |
|---|---|
| Cloud | OpenStack — Cybernuage IUT Valence |
| OS | Ubuntu 24.04 LTS (Lubuntu) |
| Flavor | m-s-v1 (2 vCPU, 16 Go RAM) |
| Réseau | iWAN — IP : 10.27.129.18 |
| Accès | VPN WatchGuard IUT (réseau 10.27.0.0/16) |
| Stockage | Disque VM standard OpenStack |

| | |
|---|---|
| Web server | Nginx 1.24 (reverse proxy SSL) |
| App server | Gunicorn 25.1.0 (2 workers sync) |
| Runtime | Python 3.12 (virtualenv) |

## 2.3 Vue réseau et sécurité du déploiement

Ports exposés (groupes de sécurité OpenStack) :

| Port | Protocole | Direction | Usage |
|---|---|---|---|
| 22 | TCP | Ingress | SSH — administration |
| 80 | TCP | Ingress | HTTP — redirection → HTTPS |
| 443 | TCP | Ingress | HTTPS — accès application |
| 5000 | TCP | Ingress | Flask debug (à fermer en production) |

Mesures de sécurité réseau :

- HTTPS avec certificat auto-signé RSA 2048 bits (OpenSSL)
- Nginx en terminaison SSL — Flask écoute uniquement sur 127.0.0.1:5000
- SSH avec PermitRootLogin yes (à durcir en production : clé SSH uniquement)
- Groupes de sécurité OpenStack limitant les ports exposés

## 2.4 API REST

L'ensemble des interactions frontend ↔ backend passe par des appels API REST en JSON :

| Méthode | Endpoint | Description |
|---|---|---|
| GET | / | Dashboard principal (ou page login si non authentifié) |
| POST | /login | Authentification utilisateur |
| GET | /logout | Déconnexion et suppression de session |
| POST | /upload | Import fichier XML (WatchGuard ou pfSense) |
| GET | /export | Téléchargement du XML actif (modifié ou original) |
| POST | /api/add_rule | Ajout d'une règle de filtrage |
| POST | /api/add_interface | Ajout d'une interface réseau ou VLAN |
| POST | /api/delete_rule | Suppression d'une règle existante |
| GET | /api/pick_source | Détection automatique du type de config source |

| | | |
|---|---|---|
| POST | /api/migrate | Lancement de la migration cross-vendor |
| POST | /api/reset_migration | Réinitialisation du wizard de migration |
| GET | /api/audit | Résultats de l'audit de sécurité |

## 2.5 Gestion des sessions

- Chaque utilisateur reçoit un UUID unique stocké dans un cookie Flask signé
- Les fichiers XML sont isolés dans uploads/{session_id}/
- Nettoyage automatique des sessions de plus de 24 heures au démarrage
- Contexte thread-local (utils.py) pour isoler les requêtes concurrentes via Gunicorn
- Deux fichiers par session : firewall.xml (original) et firewall_modifie.xml (après édition/migration)

# 3. Guide de déploiement

## 3.1 Prérequis

| Composant | Version minimale | Rôle |
|---|---|---|
| Ubuntu / Debian | 22.04+ | Système d'exploitation |
| Python | 3.12+ | Runtime application |
| Nginx | 1.18+ | Reverse proxy + SSL |
| OpenSSL | 3.0+ | Génération certificat HTTPS |
| Git | 2.x | Récupération du code source |

## 3.2 Installation pas à pas

1. Mise à jour du système et installation des dépendances :

```
apt update && apt install -y python3 python3-pip python3-venv nginx git
```

2. Récupération du code source :

```
git clone https://github.com/Ilyesse26100/NETMORPH.git /root/netmorph
```

3. Création de l'environnement virtuel et installation des packages :

```
cd /root/netmorph
python3 -m venv venv
source venv/bin/activate
pip install flask gunicorn google-generativeai lxml python-dotenv
```

## 3.3 Configuration Nginx + HTTPS

1. Génération du certificat auto-signé :

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /etc/ssl/private/netmorph.key \
  -out /etc/ssl/certs/netmorph.crt \
  -subj "/CN=netmorph"
```

2. Configuration du virtual host Nginx :

```
server {
    listen 443 ssl;
    server_name _;
    ssl_certificate /etc/ssl/certs/netmorph.crt;
    ssl_certificate_key /etc/ssl/private/netmorph.key;
    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-Proto $scheme;
```

```
        client_max_body_size 32M;
    }
}
server {
    listen 80;
    return 301 https://$host$request_uri;
}
```

3. Activation et test :

```
ln -sf /etc/nginx/sites-available/netmorph /etc/nginx/sites-enabled/
rm -f /etc/nginx/sites-enabled/default
nginx -t && systemctl restart nginx
```

## 3.4 Lancement de l'application

Lancement manuel (test) :

```
cd /root/netmorph && source venv/bin/activate
gunicorn -w 2 -b 127.0.0.1:5000 app:app --daemon
```

Service systemd pour démarrage automatique :

```
[Unit]
Description=NETMORPH Flask App
After=network.target

[Service]
User=root
WorkingDirectory=/root/netmorph
ExecStart=/root/netmorph/venv/bin/gunicorn -w 2 -b 127.0.0.1:5000 app:app
Restart=always
Environment="PATH=/root/netmorph/venv/bin"

[Install]
WantedBy=multi-user.target
```

```
sudo cp netmorph.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable netmorph
sudo systemctl start netmorph
```

## 3.5 Configuration de l'IA (optionnel)

L'auto-réparation XML utilise l'API Google Gemini. Sans clé API, l'application fonctionne normalement mais désactive cette fonctionnalité.

Créer un fichier .env à la racine du projet :

```
GEMINI_API_KEY=votre_cle_generee_sur_aistudio.google.com
```

# 4. Guide de maintenance

## 4.1 Logs et monitoring

| Source | Emplacement | Commande |
|---|---|---|
| Nginx access | /var/log/nginx/access.log | tail -f /var/log/nginx/access.log |
| Nginx errors | /var/log/nginx/error.log | tail -f /var/log/nginx/error.log |
| Gunicorn | Journal systemd | journalctl -u netmorph -f |
| Flask debug | Console (si debug=True) | python3 app.py |

## 4.2 Mise à jour du code

```
cd /root/netmorph
git pull origin main
source venv/bin/activate
pip install -r requirements.txt
sudo systemctl restart netmorph
```

## 4.3 Sauvegarde

- Code source : /root/netmorph/ (versionné sur GitHub)
- Sessions utilisateurs : /root/netmorph/uploads/ (purgées automatiquement après 24h)
- Templates XML : /root/netmorph/template/ (à inclure dans les backups)
- Configuration Nginx : /etc/nginx/sites-available/netmorph
- Certificats SSL : /etc/ssl/private/netmorph.key et /etc/ssl/certs/netmorph.crt

## 4.4 Troubleshooting courant

| Symptôme | Cause probable | Solution |
|---|---|---|
| Page blanche | Gunicorn arrêté | systemctl restart netmorph |
| 502 Bad Gateway | Flask pas sur 5000 | ss -tlnp \| grep 5000 |
| Upload échoue | Fichier > 32 MB ou pas XML | Vérifier MAX_CONTENT_LENGTH |
| Migration échoue | Template manquant | Vérifier template/Template_*.xml |
| Erreur IA | Clé Gemini manquante | Vérifier GEMINI_API_KEY dans .env |
| Onglets inactifs | Erreur JS dans le navigateur | F12 → Console pour voir l'erreur |
| HTTPS non sécurisé | Certificat auto-signé | Normal — cliquer Avancé → Continuer |

# 5. Analyse de risques — EBIOS Risk Manager

L'analyse de risques suit la méthodologie EBIOS Risk Manager (v1.5) publiée par l'ANSSI. Elle s'articule en 5 ateliers progressifs, du cadrage stratégique aux scénarios opérationnels et mesures de traitement.

## 5.1 Atelier 1 — Cadrage et socle de sécurité

### Objet de l'étude

NETMORPH — plateforme SaaS de migration de configurations firewall, hébergée sur infrastructure OpenStack de l'IUT de Valence. L'application manipule des données de configuration réseau intrinsèquement sensibles : topologie, adresses IP internes, politiques de sécurité, règles NAT.

### Valeurs métier

| ID | Valeur métier | Description |
|---|---|---|
| VM1 | Configurations firewall | Fichiers XML contenant topologie réseau, IPs, politiques de sécurité |
| VM2 | Service de migration | Disponibilité de la plateforme pour les administrateurs réseau |
| VM3 | Intégrité des migrations | Exactitude de la traduction des règles (erreur = faille de sécurité) |

### Biens supports

| ID | Bien support | Type |
|---|---|---|
| BS1 | VM OpenStack (Ubuntu 24.04, 10.27.129.18) | Infrastructure |
| BS2 | Application Flask + Gunicorn | Logiciel |
| BS3 | Serveur Nginx (reverse proxy HTTPS) | Logiciel |
| BS4 | Fichiers XML uploadés (uploads/{session_id}/) | Données |
| BS5 | Réseau IUT / VPN WatchGuard | Réseau |

### Événements redoutés

| ID | Événement redouté | Impact | Gravité |
|---|---|---|---|
| ER1 | Fuite de configurations firewall | Exposition topologie réseau et failles | 4/4 — Critique |

| | | | |
|---|---|---|---|
| ER2 | Altération d'une configuration migrée | Règles de sécurité corrompues | 4/4 — Critique |
| ER3 | Indisponibilité du service | Pas d'accès à l'outil | 2/4 — Modérée |
| ER4 | Accès non autorisé | Manipulation de configs sensibles | 3/4 — Grave |

### Socle de sécurité

- Authentification par login/mot de passe sur toutes les routes
- Sessions isolées par utilisateur (UUID aléatoire)
- HTTPS via certificat auto-signé (chiffrement en transit)
- Pare-feu OpenStack (groupes de sécurité) limitant les ports
- Nettoyage automatique des sessions de plus de 24h
- Validation XML systématique avant import (balise racine + structure)
- Pas de stockage permanent des configurations client

## 5.2 Atelier 2 — Sources de risque

| ID | Source de risque | Motivation | Objectif visé | Pertinence |
|---|---|---|---|---|
| SR1 | Attaquant externe | Exfiltration de configs pour cartographier un réseau | OV1 — Accéder aux fichiers XML | Élevée |
| SR2 | Utilisateur malveillant interne | Curiosité, sabotage | OV2 — Accéder aux sessions d'autres utilisateurs | Modérée |
| SR3 | Attaquant opportuniste | Exploitation de vulnérabilités | OV4 — Compromettre la VM comme rebond | Modérée |

## 5.3 Atelier 3 — Scénarios stratégiques

### SS1 : Exfiltration via accès réseau

Chemin d'attaque : SR1 → exploite une vulnérabilité du VPN IUT → accède à 10.27.129.18 → bypasse l'authentification Flask → exfiltre les XML des sessions actives.

Parties prenantes : Infrastructure VPN IUT (menace modérée)

**Gravité : Critique (4/4)**

### SS2 : Escalade de privilèges interne

Chemin d'attaque : SR2 → se connecte légitimement → énumère les UUID de sessions → accède aux fichiers d'un autre utilisateur via traversée de chemin.

**Gravité : Grave (3/4)**

### SS3 : Compromission de la VM

Chemin d'attaque : SR3 → scanne les ports ouverts → exploite SSH ou une dépendance Python vulnérable → prend le contrôle de la VM → accès total.

**Gravité : Critique (4/4)**

## 5.4 Atelier 4 — Scénarios opérationnels

### SO1 : Exploitation de l'authentification faible

Actions élémentaires :

- Brute force sur /login (absence de rate limiting)
- Interception du cookie de session (absence de flag Secure/HSTS)
- Accès aux routes API avec le cookie volé
- Téléchargement des XML via /export

**Vraisemblance : Élevée**

### SO2 : Path traversal sur les sessions

- Authentification légitime
- Manipulation du paramètre session_id
- Accès aux fichiers uploads/{autre_session_id}/firewall.xml

**Vraisemblance : Modérée**

(le session_id est un UUID dans un cookie signé Flask, difficile à deviner)

### SO3 : Exploitation d'une CVE

- Scan Nmap de la VM (ports 22, 80, 443, 5000)
- Identification des versions Nginx/Python/Flask
- Exploitation d'une vulnérabilité connue (CVE)
- Reverse shell → accès root

**Vraisemblance : Modérée**

## 5.5 Atelier 5 — Traitement du risque

Mesures de sécurité identifiées et leur état d'avancement :

| Risque | Mesure de traitement | Priorité | Statut |
|---|---|---|---|
| SO1 — Brute force | Implémenter rate limiting (flask-limiter) | Haute | À faire |
| SO1 — Cookie | Ajouter HSTS + flag Secure sur cookies | Haute | À faire |
| SO1 — Mot de passe | Hash bcrypt + complexité imposée | Moyenne | À faire |
| SO2 — Path traversal | Validation session_id == session courante | Haute | Fait (sid()) |
| SO3 — CVE | Automatiser apt upgrade + pip audit | Moyenne | À faire |
| Général | Remplacer certificat par Let's Encrypt | Basse | Bloqué (IP interne) |
| Général | Journalisation des accès (who/what/when) | Moyenne | À faire |
| Général | Snapshots automatiques VM OpenStack | Basse | À faire |

## Risques résiduels acceptés

- Certificat auto-signé → avertissement navigateur (acceptable en contexte IUT)
- Authentification basique (un couple login/mdp partagé) → acceptable pour un PoC
- Fichiers XML non chiffrés au repos → acceptable car nettoyage automatique 24h
- Port 5000 ouvert (debug) → à fermer en production

# 6. Annexes

## 6.1 Arborescence du projet

```
netmorph/
├── app.py                  # Point d'entrée Flask
├── engine.py               # Moteur de migration + IA
├── parser.py               # Parsing XML configs
├── ui_template.py          # Interface HTML/CSS/JS
├── audit.py                # Audit de sécurité
├── utils.py                # Helpers sessions/chemins
├── requirements.txt        # Dépendances Python
├── logo.png                # Logo NETMORPH
├── netmorph.ico            # Icône
├── .env                    # Clé API Gemini (non versionné)
├── template/
│   ├── Template_Firebox.xml
│   └── Template_pfsense.xml
├── uploads/                # Sessions utilisateurs (auto-purgé 24h)
│   └── {uuid}/
│       ├── firewall.xml
│       └── firewall_modifie.xml
└── docs/                   # Documentation technique
```

## 6.2 Dépendances Python

| Package | Version | Usage |
|---|---|---|
| Flask | 3.1.3 | Framework web WSGI |
| Gunicorn | 25.1.0 | Serveur WSGI production |
| lxml | 6.0.2 | Parsing/manipulation XML avancé |
| google-generativeai | 0.8.6 | Auto-réparation XML via Gemini |
| python-dotenv | 1.2.2 | Chargement variables d'environnement |

## 6.3 Identifiants par défaut

| Service | Login | Mot de passe | Remarque |
|---|---|---|---|
| Application web | iutuser | iutval | Défini dans app.py (USERS dict) |
| SSH (VM) | root | (défini manuellement) | passwd root sur la VM |

**⚠ En production, changer impérativement les identifiants par défaut et implémenter un système d'authentification robuste (LDAP, SSO, bcrypt).**